Process Verification

Final Project

Quentin Meneghini

20th May 2026

Facultat d'Informàtica de Barcelona

Contents

Chapter 1

Design Overview

1.1 Overview

The `gcd` module computes the Greatest Common Divisor of two unsigned integers using the subtractive Euclidean algorithm. It processes one input pair at a time and communicates through two independent ready/valid handshake channels.

Mathematical Definition

For unsigned operands A and B:

$$\gcd(A, 0) = A$$

$$\gcd(0, B) = B$$

$$\gcd(0, 0) = 0 \quad \text{(covers both above)}$$

$$\gcd(A, A) = A$$

$$\gcd(A, B) = \gcd(A - B, B) \quad \text{when } A > B$$

$$\gcd(A, B) = \gcd(A, B - A) \quad \text{when } B > A$$

Operation Sequence

1. **IDLE:** The module waits in the `IDLE` state, asserting `in_ready = 1`.
2. **Handshake & RUN:** On the input handshake, operands are latched. The result is determined immediately for zero or equal operands; otherwise, iterative subtraction begins

in the RUN state.

3. **DONE:** When convergence is reached, the result is latched into the output register and the module enters the DONE state, asserting out_valid = 1.

4. **Return:** After the output handshake completes, the module returns to IDLE.

1.2 Parameters

Parameter	Default	Description
WIDTH	16 (must be ≥ 1)	Width of the data operands

Table 1.1: Module Parameters

1.3 Top-Level Interface

Clock and Reset

Signal	Direction	Width	Description
clk	Input	1	Clock signal
rst_n	Input	1	Active-low sync reset

Input Channel

Signal	Direction	Width	Description
in_valid	Input	1	Asserted by the producer when a_in and b_in hold valid data
in_ready	Output	1	Asserted by the module when it can accept a new transaction. High only in the IDLE state.
a_in	Input	WIDTH	First operand
b_in	Input	WIDTH	Second operand

Output Channel

Signal	Direction	Width	Description
out_valid	Output	1	Asserted by the module when gcd_out holds a valid result. High for the entire DONE state.
out_ready	Input	1	Asserted by the consumer when it is ready to accept the result.
gcd_out	Output	WIDTH	GCD result. Valid and stable for the entire period that out_valid is asserted.

1.4 Functional Description

The behavior of the `gcd` module is driven by a Finite State Machine (FSM) comprising three states:

FSM States

State	in_ready	out_valid	Description
IDLE	1	0	Waiting for input. Ready to accept a new transaction.
RUN	0	0	Iterative subtraction in progress. No new input accepted.
DONE	0	1	Result ready. Waiting for the consumer to accept it.

Algorithm Execution (RUN State)

Each clock cycle while in the RUN state, the working registers are updated combinatorially for the next cycle:

- **If** `a_reg > b_reg`: `a_next = a_reg - b_reg` and `b_next = b_reg`.
- **Else if** `b_reg > a_reg`: `b_next = b_reg - a_reg` and `a_next = a_reg`.

State Transitions

IDLE → DONE: Triggered by an input handshake **AND** (`a_in == 0 OR b_in == 0 OR a_in == b_in`). The result is known immediately, making this a 1-cycle transition.

IDLE → RUN: Triggered by an input handshake **AND** (`a_in != 0 AND b_in != 0 AND a_in != b_in`).

RUN → DONE: Triggered when the next-cycle values of the working registers are equal (`a_next == b_next`). This evaluates against the combinatorial *next-cycle* values, firing on the same clock edge as the final subtraction step.

DONE → IDLE: Triggered by a successful output handshake (`out_valid == 1 AND out_ready == 1`).

Chapter 2

Test Plan

2.1 Overview

Features Under Test

The features to be tested are divided into tree groups. The idea is to test both normal execution and edge cases for each group.

- **[C] Control dependent (Consumer/Producer)** Reset behavior, Input ready/valid handshake/stall, Output ready/valid handshake/stall.
- **[D] Data dependent (Input/output)** edge cases (primes/coprimes, multiples, min value, max value), early exits (zero operands, equal operands), and correct output.
- **[S] State machine** Transitions between `IDLE`, `RUN`, and `DONE`, along with their cycle timings.

Tool for Testing

To test these features we will use the following tools:

- **[F] FORMAL**: here we will verify the logic from inside the dut. We will use a bounded verification tool. The analysis will touch the expected control output and the FSM states, hold, and transitions.
- **[U] UVM**: here we will verify the logic as a black box through stimulus. Using a golden model we will verify the correctness of the data.

- [**H**] **HARDWARE**: here we will verify the hardware timing and correct timings. Assertions based approach is used in a formal model.

2.2 Traceability Matrices

Here we introduce all the requirements extracted from the specification. It is divided in the previously presented features and sub-features accompanied by the tool used to verify it.

Req ID	Feature	Sub-Feature	Description	Tool
REQ_000	C	Reset	Correct init and graceful recover from reset mid-transaction	F/H/U
REQ_001	C	Input accept	Producer `in_valid` is high and DUT `in_ready` is high accepts input	F
REQ_002	C	Input stall	DUT `in_ready` is low and producer `in_valid` is high	F
REQ_003	C	Output accept	Consumer `out_ready` is high and DUT `out_valid` is high accepts output	F
REQ_004	C	Output stall	Consumer `out_ready` is low and DUT `out_valid` is high	F/H
REQ_005	C	Output stability	`gcd_out` is stable while `out_valid` is high	F/H
REQ_006	D	All operands	Correct `gcd_out` for general operands	U
REQ_007	D	Edge cases	Correct `gcd_out` for primes/coprimes, multiples, and min/max values	U
REQ_008	D	Zero operands	Correct `gcd_out` for $a = 0$ and/or $b = 0$, early exit	F/H/U
REQ_009	D	Equal operands	Correct `gcd_out` for $a = b$, early exit	F/H/U
REQ_010	S	States	FSM state is either `IDLE`, `RUN`, or `DONE`	F
REQ_011	S	State hold	FSM holds its state strictly for the necessary duration	F
REQ_012	S	Transitions	FSM transitions following the specification	F
REQ_013	S	Assigned signal	FSM determines output signals (`in_valid`, `out_ready`)	F

This second table is meant to associate each tool and relative test to the requirement it tries to verify. A description explains what the test is actually looking for.

Tool	Test name	Description	REQ ID
F	`inputs`	Assume a compliant producer that holds `in_valid` and data stable during stalls	REQ_002
F	`_reset`	Prove DUT initializes to `IDLE` and clears all registers on reset	REQ_000
F	`_state`	Prove FSM strictly operates within valid, mutually exclusive states	REQ_010
F	`idle_idle`, `run_run`, `done_done`	Prove FSM correctly holds current state when transition conditions are not met, including output stalls	REQ_004, REQ_011
F	`idle_done`, `idle_run`, `run_done`, `done_idle`	Prove FSM accurately transitions between states based on handshakes and data path evaluations	REQ_001, REQ_003, REQ_008, REQ_009, REQ_012
F	`_in_ready`, `_out_valid`	Prove control output signals are strictly tied to their respective FSM states	REQ_013
F	`outputs`	Prove `gcd_out` stability while `out_valid` is asserted	REQ_005
H	`chk_out_valid_no_drop`	Check that the DUT does not drop `out_valid` while waiting for the consumer `out_ready` to assert	REQ_004
H	`chk_out_data_stable`	Check that the DUT holds `gcd_out` stable while `out_valid` is asserted	REQ_005
H	`chk_reset`	Check that in case of reset the DUT clear all of its outputs	REQ_000
H	`chk_early_exit`	Check that the DUT skips to the `DONE` state in case of early exit	REQ_008, REQ_009
U	`scb.write_rst`	Verify that the golden model resets its internal state mid-transaction	REQ_000
U	`scb.write_in`, `scb.write_out`	Verify that the golden model output is equal to the DUT `gcd_out`	REQ_006, REQ_007, REQ_008, REQ_009

2.3 Target

Coverage

The target coverage is just a measure of what we would like to tackle in this test plan for each area of the dut. In practice the objectives are a complete (100%) coverage of the functional description and possibly a complete (100%) code coverage.

Stimulus

The stimulus is divided into multiple tests that tackle different scenarios.

- **gcd_bringup_test**: Test the basic functionality of the verification system as a whole
- **gcd_det_test**: Test the edge cases and tries to obtain the maximum coverage with the minimal number of tests
- **gcd_rnd_test**: Test the DUT with random instructions to find edge cases that have not been tested otherwise

2.4 Diagrams

This section will introduce the FSM in form of a diagram. Every nameless arrow implies that all inputs that have not been specified translate in that state. The plan for the UVM architecture is displayed in the Implementation chapter 3

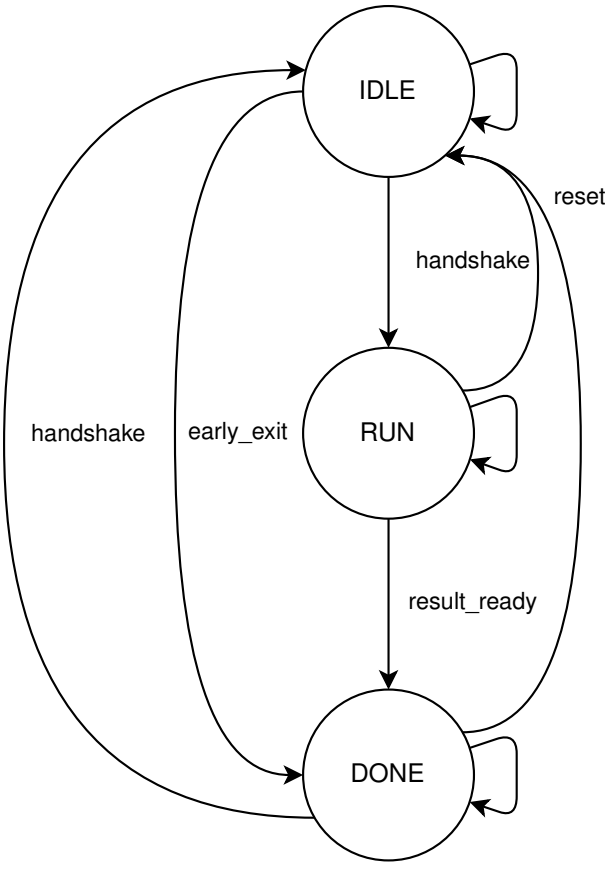

Figure 2.1: Finite State machine of the DUT

Chapter 3

Implementation

3.1 FORMAL

EBMC tool has been used to perform the bounded verification of this module. Here the assumptions are the initial `rst_n` low and the stability of inputs while `in_valid` is high. The assertions are as follows:

`_reset` Verify that asserting a reset unconditionally forces the FSM back to the `IDLE` state on the next cycle with all registers cleared.

```
155     _reset: assert property (
156         @(posedge clk)
157         !rst_n |=>
158             (state == IDLE) &&
159             (a_reg == '0) &&
160             (b_reg == '0) &&
161             (result_reg == '0) &&
162             (result_reg == '0)
163     );
```

`_state` Guarantees the FSM is mathematically restricted to the five valid encoded states, proving no illegal states can be reached.

```
165     _state: assert property (
166         @(posedge clk)  disable iff(!rst_n)
167         (state == IDLE) ||
168         (state == RUN)  ||
169         (state == DONE)
170     );
```

State Hold (`idle_idle`, `run_run`, `done_done`): Guarantee that the FSM retains its current state when no progression signal or abort is asserted.

```
172     idle_idle: assert property (
173         @(posedge clk) disable iff(!rst_n)
174         (state == IDLE) && !(in_ready && in_valid) |=>
175             (state == IDLE)
176     );
177
178     run_run: assert property (
179         @(posedge clk) disable iff(!rst_n)
180         (state == RUN) && !(a_next == b_next) |=>
181             (state == RUN)
182     );
183
184     done_done: assert property (
185         @(posedge clk) disable iff(!rst_n)
186         (state == DONE) && !(out_ready && out_valid) |=>
187             (state == DONE)
188     );
```

State Progression (`idle_done`, `idle_run`, `run_done`, `done_idle`): Prove that the FSM correctly transitions to the next expected state when the appropriate progression flag is asserted.

```
190     idle_done: assert property (
191         @(posedge clk) disable iff(!rst_n)
192         (state == IDLE) && in_ready && in_valid &&
193         (a_in == 0 || b_in == 0 || a_in == b_in) |=>
194             (state == DONE)
195     );
196
197     idle_run: assert property (
198         @(posedge clk) disable iff(!rst_n)
199         (state == IDLE) && in_ready && in_valid &&
200         !(a_in == 0 || b_in == 0 || a_in == b_in) |=>
201             (state == RUN)
202     );
203
204     run_done: assert property (
205         @(posedge clk) disable iff(!rst_n)
206         (state == RUN) && (a_next == b_next) |=>
207             (state == DONE)
208     );
209
210     done_idle: assert property (
211         @(posedge clk) disable iff(!rst_n)
212         (state == DONE) && out_ready && out_valid |=>
213             (state == IDLE)
214     );
```

Output stability (`outputs`): Verify that the results remain stable until the handshake happens.

```
226     outputs: assert property (
227         @(posedge clk) disable iff(!rst_n)
228         out_valid |=>
229             $stable(gcd_out)
230     );
```

Handshakes (_out_valid, _in_ready): Prove that the flags are never asserted spuriously and that they follow the FSM transitions.

```
216     _out_valid: assert property (
217         @(posedge clk) disable iff(!rst_n)
218         (state == DONE) == out_valid
219     );
220
221     _in_ready: assert property (
222         @(posedge clk) disable iff(!rst_n)
223         (state == IDLE) == in_ready
224     );
```

3.2 HARDWARE

The hardware checking happens inside the DUT interface and acts as a raw signal controller. To do so, 5 properties with arguments to reuse them if necessary were created.

```
35      property p_valid_no_drop(valid, ready);
36          @(posedge clk) disable iff (
37              !rst_n ||
38              $isunknown(valid) ||
39              $isunknown(ready))
40          valid && !ready
41              |=> valid;
42      endproperty
43
44      property p_data_stable(valid, ready, data);
45          @(posedge clk) disable iff (
46              !rst_n ||
47              $isunknown(valid) ||
48              $isunknown(ready) ||
49              $isunknown(data))
50          valid && !ready
51              |=> $stable(data);
52      endproperty
53
54      property p_early_exit;
55          @(posedge clk) disable iff (
56              !rst_n ||
57              $isunknown(in_valid) ||
58              $isunknown(in_ready))
59          (in_valid && in_ready) &&
60          (a_in == 0 || b_in == 0 || a_in == b_in)
61              |=> out_valid;
62      endproperty
63
64      property p_reset;
65          @(posedge clk)
66          !rst_n
67              |=> in_ready && !out_valid && (gcd_out == 0);
68      endproperty
69
70      property p_handshake(valid, ready);
71          @(posedge clk) disable iff (!rst_n)
72          valid && ready;
73      endproperty
```

These properties were used to make 7 assertions and 2 covers. 3 assertions are only used to control UVM drivers correctness.

`chk_reset` Checks that in case of reset the DUT clear all of its outputs.

`chk_early_exit` Checks that the DUT skips to the `RUN` state in case of early exit.

`chk_out_valid_no_drop` Checks that `out_valid` is not dropped unless a handshake happens.

`chk_out_data_stable` Checks that `gcd_out` is stable while `out_valid` is asserted.

The 2 covers simply make sure the handshakes, both input and output are covered.

Eventual Finish

To Guarantee that a transaction finish in reasonable time every time an input is accepted a timer is used. This timer count how many cycles that input takes to be processed and if it values it to be too many returns an error. The idea is to assure that a transaction always ends. The System Verilog sequence `##[1:$]` was unusable since the UVM timeout would have destroyed its usefulness. And using a fixed maximum would make this test very limited and somewhat inefficient.

To do so we made 3 components and added them to the interface:

- **Estimation** a function that make an estimation of the execution time

```
104     function automatic int unsigned exp_cycles(
105         bit [DATA_WIDTH-1:0] a,
106         bit [DATA_WIDTH-1:0] b
107     );
108         int unsigned cycles = 0;
109 
110         if (a == 0 || b == 0 || a == b) return 2;
111 
112         while (a != 0 && b != 0 && a != b) begin
113             if (a > b) begin
114                 cycles += (a / b);
115                 a = a % b;
116             end else begin
117                 cycles += (b / a);
118                 b = b % a;
119             end
120         end
121 
122         return cycles;
123     endfunction
```

- **Timer** a thread function that time the execution

```
125     task automatic timer(
126         bit [DATA_WIDTH-1:0] a,
127         bit [DATA_WIDTH-1:0] b
128     );
129         int unsigned max_cycles = exp_cycles(a, b);
130         int unsigned count = 0;
131 
132         while (count <= max_cycles + MARGIN_TIMEOUT) begin
133             @(posedge clk);
134             if (out_valid || !rst_n) return;
135             count++;
136         end
137 
138         `uvm_error("CHK", $sformatf(
139             "[TIMEOUT] [A] %0d [B] %0d [EXPECTED_CYCLES] %0d [WAITED_CYCLES
                ] %d",
140             a, b, max_cycles, max_cycles + MARGIN_TIMEOUT))
141     endtask
```

- **Spawner** a thread spawner for each accepted input

```
143     always @(posedge clk) begin
144         if (rst_n && in_valid && in_ready) begin
145             fork
146                 timer(a_in, b_in);
147             join_none
148         end
149     end
```

3.3 UVM

The UVM architecture is organized as shown in the following diagram. Details of salient components are explained in the following subsections.

Transactions

3 transaction types have been defined: input, output, and reset. The transactions are used both by the driver and the monitor, this leads to a less modular but easier to modify code with minimal overhead. The transactions are stripped of any unnecessary information like the handshake flags which are completely handled by the driver. The variable `delay_cycles` is added to both input and output transactions to enable the test with some timing controls.

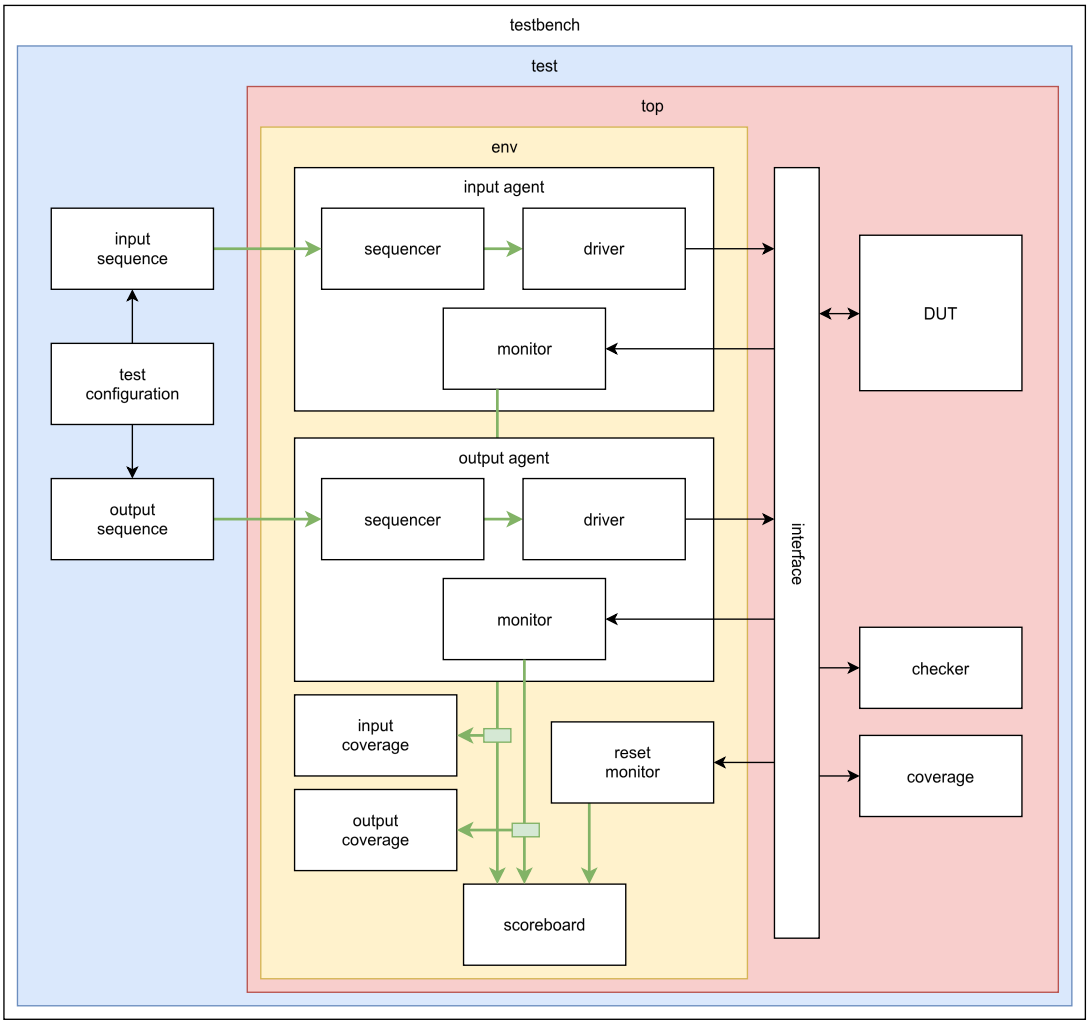

Figure 3.1: UVM architecture of the DUT

Drivers

Each I/O driver need to:

1. handle the handshake protocol
2. apply the cycles delay
3. apply the signals (if any)
4. release handshake

They are also enabled to handle the initial reset as well as mid-transaction reset.

Scoreboard

The scoreboard keep track of each input and wait for the output to confront it with an internal calculation. In case of reset mid-transaction the transaction is considered a success and the

simulation continues. The calculation uses a different model for the calculation of the gcd, modulo instead of subtraction, for simplicity of implementation and performance.

```
523         function bit [DATA_WIDTH-1:0] compute_gcd(
524             bit [DATA_WIDTH-1:0] a,
525             bit [DATA_WIDTH-1:0] b
526         );
527             if (a == 0 || b == 0) return (a == 0) ? b : a;
528 
529             while (b != 0) begin
530                 bit [DATA_WIDTH-1:0] temp = b;
531                 b = a % b;
532                 a = temp;
533             end
534 
535             return a;
536         endfunction
```

Coverage

Apart from the code coverage that is already handled by Questa, some functional coverage was added to complete the coverage of the DUT. They can be divided into two groups:

- **CONTROL** this is tackled by Hardware assertions and coverage of the successful handshake protocols. It is important to note that Assertions are used as coverage to remove a repetition in the final report (assertion + coverage).
- **DATA** this is tackled by the UVM and further divided into 2 component that analyze inputs and output.
  - **Input** here we verify the edge cases and combination of them.
  - **Output** here we verify the edge cases but from the output perspective.

Tests

Although the architecture is mostly modular, and it's possible to build whatever test we might need, 3 tests have been implemented. (Described in the Test Plan 2) Each test sequence is wrapped into a virtual sequence that contains both driver's information.

Chapter 4

Results

Overall, the DUT was verified to be functionally correct. Both the formal verification tool and the UVM architecture confirm this result. However, the code coverage analysis revealed some minor structural inefficiencies in the implementation. While these do not endanger the functionality of the DUT, they might lead to unoptimized results upon synthesis.

4.1 Coverage Analysis

From the coverage report, we can observe that the deterministic test vectors completely cover the functional specification (100%). To clarify the functional metrics:

- `COVERGROUP` refers to the UVM coverage blocks.
- `DIRECTIVE` refers to the two cover properties in the Hardware coverage.
- `ASSERTION` refers to the Hardware-based assertions.

The Total Coverage score calculates the average of these three functional coverage categories along with the seven code coverage points (detailed below in Table 4.1).

```
TOTAL COVERGROUP COVERAGE: 100.00%  COVERGROUP TYPES: 2
TOTAL DIRECTIVE COVERAGE: 100.00%  COVERS: 2
TOTAL ASSERTION COVERAGE: 100.00%  ASSERTIONS: 7
Total Coverage By Instance (filtered view): 94.68%
```

The Total Coverage score is slightly reduced by a few specific structural loopholes identified

during the code coverage extraction:

- **Branch [95.45%]**: Contains 1 uncovered bin.
  - *Line 53*: A branch is never evaluated as false. This is the consequence of a concatenation of conditions that makes the false outcome logically unreachable. The `if` statement could theoretically be removed.
- **Condition [71.42%]**: Contains 2 uncovered bins.
  - *Line 53*: The condition is never false. Similar to the branch issue, previous conditions make the false outcome unreachable, making the statement redundant.
  - *Line 75*: Part of the condition (`in_ready`) is never false. This happens because a preceding condition (`state == IDLE`) already forces this signal high, making the false evaluation unreachable.
- **FSM Transitions [80.00%]**: Contains 1 uncovered bin.
  - *Line 63*: The state machine never transitions directly from `RUN` to `IDLE`. This occurs because the reset signal was only exercised during the initial initialization of the DUT, and an active reset is the only mechanism capable of producing this specific mid-transaction transition.

Enabled Coverage	**Bins**	**Hits**	**Misses**	**Coverage**
Branches	22	21	1	95.45%
Conditions	7	5	2	71.42%
Expressions	2	2	0	100.00%
FSM States	3	3	0	100.00%
FSM Transitions	5	4	1	80.00%
Statements	25	25	0	100.00%
Toggles	527	527	0	100.00%

Table 4.1: Structural Code Coverage Summary

In the end we consider the code coverage complete as FSM transitions are completely covered by the Formal verification and the other uncovered bins are unreachable code.

4.2 Bug

I introduced a bug in the DUT that should give wrong results as the formula is completely wrong. We expect the DUT to give wrong gcd for operands that do not fall in the early exit conditions. Here is the code diff:

```
always_comb begin
    a_next = a_reg;
    b_next = b_reg;
    if (state == RUN) begin
-       if (a_reg >b_reg)
+        if (a_reg <b_reg)
            a\_next = a\_reg - b\_reg
-       else if (b_reg >a_reg)
+        else if (b_reg <a_reg)
            b\_next = b\_reg - a\_reg;
    end
end
```

Analysis

Appendix A

Appendix Chapter

A.1 Use of tools

- Visual Studio Code: systemverilog IDE.
- QuestaSim: CLI tester.
- GTKwave: wave visualizer.

A.2 Use of time

A total of h distributed as follows.

- Understanding the problem: 2h.
- Planning 6h.
- Implementation: formal 2h, hardware 2h, uvm 8h.
- Coverage analysis 3h.
- Report: 7h.

A.3 Use of AI

- Gemini (Pro): sentence enhancing, bug fixer, report enhancing